Deployment Guide

Ezitech Engineering Framework — Case Study AI-005

Internship Performance Prediction & Risk Analytics Platform

Prepared by: Sadaf Riaz & Saira Ejaz

1. Deployment Overview

The platform ships as a set of local services orchestrated with Docker Compose (PostgreSQL, Redis, MLflow, Prometheus, Grafana) plus a FastAPI application that runs directly on the host via Uvicorn. There is no separate frontend build step — the dashboards in app/static/ are served directly by FastAPI as static files, so "deploying the app" and "deploying the API" are the same step.

Layer	Technology	Runs via
API + dashboards	FastAPI, served with Uvicorn	Host machine (venv)
Database	PostgreSQL 16	Docker container: intern_postgres
Cache	Redis 7	Docker container: intern_redis
Experiment tracking	MLflow	Docker container: intern_mlflow
Metrics	Prometheus	Docker container: intern_prometheus
Dashboards (ops)	Grafana	Docker container: intern_grafana

2. Prerequisites

- Docker Desktop (Compose v2) for Postgres, Redis, MLflow, Prometheus, Grafana
- Python 3.12
- Git
- (Optional) DBeaver — for browsing Postgres tables visually

3. Step-by-Step Setup

3.1 Clone the repository

```
git clone https://github.com/2023-BSE-057-Saira/intern-analytics-platform.git
cd intern-analytics-platform
```

3.2 Start infrastructure services

Postgres, Redis, MLflow, Prometheus, and Grafana are all defined in docker-compose.yml. Start them with:

```
docker-compose up -d
```

Postgres auto-loads every .sql file in the sql/ directory the first time its volume is created (docker-entrypoint-initdb.d), so a brand-new environment gets schema.sql, auth_migration.sql, student_features_migration.sql, and mentor_review_migration.sql applied automatically in one pass. This only happens on first creation of the postgres_data volume — see Section 5 for applying new migrations against an existing database.

Verify all five containers are healthy:

```
docker ps
```

Expect to see intern_postgres, intern_redis, intern_mlflow, intern_prometheus, and intern_grafana all with status "Up".

3.3 Set up the Python environment

```
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate     # macOS / Linux
python -m pip install -r requirements.txt
```

3.4 Configure environment variables

```
copy .env.example .env       # Windows
cp .env.example .env         # macOS / Linux
```

The defaults in .env.example already match docker-compose.yml's credentials (DATABASE_URL, REDIS_URL, MLFLOW_TRACKING_URI), so no edits are required for a local run.

3.5 Generate data and train models (first-time setup only)

A fresh database has schema but no rows. Run the pipeline scripts in app/ml/ in this order to populate synthetic intern data and train the prediction models:

```
python app/ml/generate_synthetic_data.py
python app/ml/build_dataset.py
python app/ml/build_performance_dataset.py
python app/ml/build_success_dataset.py
python app/ml/build_growth_dataset.py
python app/ml/build_weekly_performance.py
python app/ml/train_dropout_model.py
python app/ml/train_performance_model.py
python app/ml/train_success_model.py
python app/ml/train_growth_model.py
python app/ml/generate_evaluation_report.py
```

Trained models are written to app/ml/saved_models/. This step can be skipped if you're pulling a database dump that already includes generated data (see Section 6).

3.6 Seed demo user accounts

Login accounts (admin, mentor, and student roles) are separate from intern/mentor records and are created by:

```
python scripts/seed_users.py
```

This creates one login per intern/mentor row using bcrypt-hashed passwords, for 800 interns and 30 mentors.

3.7 Run the API

```
uvicorn app.main:app --reload
```

Endpoint	URL
Landing page	http://localhost:8000/
Login	http://localhost:8000/login
API docs (Swagger)	http://localhost:8000/docs
Health check	http://localhost:8000/health

4. Supporting Services

Service	URL	Credentials
API Docs	http://localhost:8000/docs	—
MLflow	http://localhost:5000	—
Prometheus	http://localhost:9090	—
Grafana	http://localhost:3000	admin / admin123
Postgres	localhost:5432	intern_admin / intern_pass123

Connect to Postgres from DBeaver using the credentials above to browse tables visually.

5. Applying Migrations to an Existing Database

docker-entrypoint-initdb.d only runs once, the first time the postgres_data volume is created. On any environment where the containers already existed before a new migration file was added (for example, after pulling mentor_review_migration.sql), that migration must be applied by hand against the running container:

```
# Windows PowerShell (psql is not required on the host — exec into the container instead)
Get-Content sql/mentor_review_migration.sql | docker exec -i intern_postgres `
  psql -U intern_admin -d intern_analytics

# macOS / Linux
docker exec -i intern_postgres psql -U intern_admin -d intern_analytics < sql/mentor_review_migration.sql
```

Alternatively, open the .sql file's contents in DBeaver's SQL editor (connected using the Postgres credentials above) and execute it directly — this avoids any local psql/CLI setup entirely and is the simplest option on Windows.

After a migration, restart Uvicorn so SQLAlchemy's session state doesn't reference stale columns.

6. Restoring from a Database Dump

To skip synthetic data generation and model training (Section 3.5) and start from an already-populated database, restore data_export.dump after the containers are up:

```
docker cp data_export.dump intern_postgres:/tmp/data_export.dump
docker exec -it intern_postgres pg_restore -U intern_admin -d intern_analytics --clean --if-exists
/tmp/data_export.dump
```

7. Troubleshooting

Symptom	**Likely Cause**	**Fix**
Dashboard loads but shows no data / silent 500s	A migration adding a column the ORM models now expect (e.g. reviewed on weekly_reports) hasn't been run against this database yet	Apply the missing migration per Section 5, then restart Uvicorn
psql: command not found (Windows PowerShell)	Postgres client tools aren't installed on the host — this is expected, since Postgres itself runs inside Docker	Use the docker exec approach in Section 5 instead of a local psql install
docker ps shows fewer than 5 containers	One service failed to start, often a port already in use (5432, 6379, 5000, 9090, 3000)	Run docker-compose logs <service> to see the error; stop the conflicting local service or change the port mapping
uvicorn starts but /predict/* endpoints error	Model training (Section 3.5) hasn't been run, so app/ml/saved_models/ is empty	Run the training scripts in Section 3.5 in order
Login fails for every account	scripts/seed_users.py hasn't been run against this database	Run python scripts/seed_users.py